

Stratégie de tests — Kasa

1. Outils

Niveau	Outil	Justification
Unitaire / intégration composants	Jest + React Testing Library (fournis par <code>react-scripts test</code>)	Déjà configurés par Create React App, aucune dépendance supplémentaire nécessaire
End-to-end	Playwright (@ <code>playwright/test</code>)	Déjà en dépendance ; couvre les parcours réels dans un vrai navigateur, y compris le fichier <code>public/logements.json</code> servi tel quel

Pas de Vitest (migrerait hors de CRA sans bénéfice ici) ni de Cypress (redondant avec Playwright déjà présent).

2. Organisation des fichiers

```
src/ components/ Card.jsx Card.test.jsx Collapse.jsx Collapse.test.jsx Slideshow.jsx
Slideshow.test.jsx pages/ Homepage.js Homepage.test.js RentalDetails.js RentalDetails.
test.js e2e/ navigation.spec.js playwright.config.js src/setupTests.js # jest-dom +
polyfills TextEncoder/TextDecoder (requis par react-router-dom v7)
```

Convention : `*.test.js/*.test.jsx` colocalisé avec le fichier testé (détecté automatiquement par `react-scripts test`, aucune config à ajouter). Les specs Playwright vivent à part, dans `e2e/`, pour ne pas être exécutées par Jest et inversement.

3. Ce qui est couvert aujourd'hui

Unitaire / intégration (`yarn test`)

Fichier	Ce qui est vérifié
<code>Card.test.jsx</code>	Rendu du titre, de l'image, et du lien vers <code>/location/:id</code>

<code>Collapse.test.jsx</code>	Texte masqué par défaut, affiché après clic sur le chevron ; rendu en liste de paragraphes pour <code>tag="equipments"</code>
<code>Slideshow.test.jsx</code>	Affichage de la première photo au montage, avance/recul du carrousel avec bouclage aux extrémités, placeholder si aucune photo, message d'erreur si le fetch échoue
<code>Homepage.test.js</code>	Une carte par logement récupéré, message d'erreur si le fetch échoue
<code>RentalDetails.test.js</code>	Affichage des informations du logement (dont la conversion note → étoiles), redirection vers <code>/error</code> si l'ID est inconnu, message d'erreur si le fetch échoue

Le service `src/services/rentals.js` est mocké dans ces tests (`jest.mock`) pour isoler la logique de rendu du réseau ; `findRentalById` reste la vraie implémentation (via `jest.requireActual`) pour vérifier le comportement réel de résolution par ID.

End-to-end (`yarn test:e2e`)

`e2e/navigation.spec.js` couvre, dans un vrai navigateur avec le vrai `public/logements.json` :

1. Accueil → clic sur une carte → page de détail affichée.
2. URL avec un ID de logement inexistant → redirection vers la page 404.
3. Navigation vers "À propos" sans rechargement complet.

4. Priorités pour la suite

Non couvert actuellement — à ajouter en priorité si le projet évolue :

- **Header.js** : classe `active` selon `currentPage`, présence des liens `Link`.
- **About.js** : rendu des 4 blocs `Collapse`.
- **Footer.js** : composant statique, priorité basse.
- Cas limite `Slideshow` avec exactement 1 photo (branche spécifique du composant).
- Test e2e couvrant l'accessibilité clavier du chevron `Collapse` (`onKeyDown` Entrée/Espace).

5. Stratégie de couverture

Pas d'objectif de couverture à 100 % — prioriser la logique métier (résolution d'ID, calcul des étoiles, rotation du carrousel, gestion d'erreur) plutôt que les composants purement présentationnels (`Footer`).

```
yarn test -- --coverage --watchAll=false
```

Cible réaliste : 60–70 % sur `src/pages` et `src/components`. Ne pas chercher à couvrir `src/styles`, les assets, ni `index.js`.

6. Commandes utiles

```
yarn test # Jest en mode watch CI=true yarn test --watchAll=false # Jest, une seule p  
asse (CI) yarn test:e2e # Playwright (démarré le serveur de dev automatiquement) yarn  
lint # ESLint sur src/
```